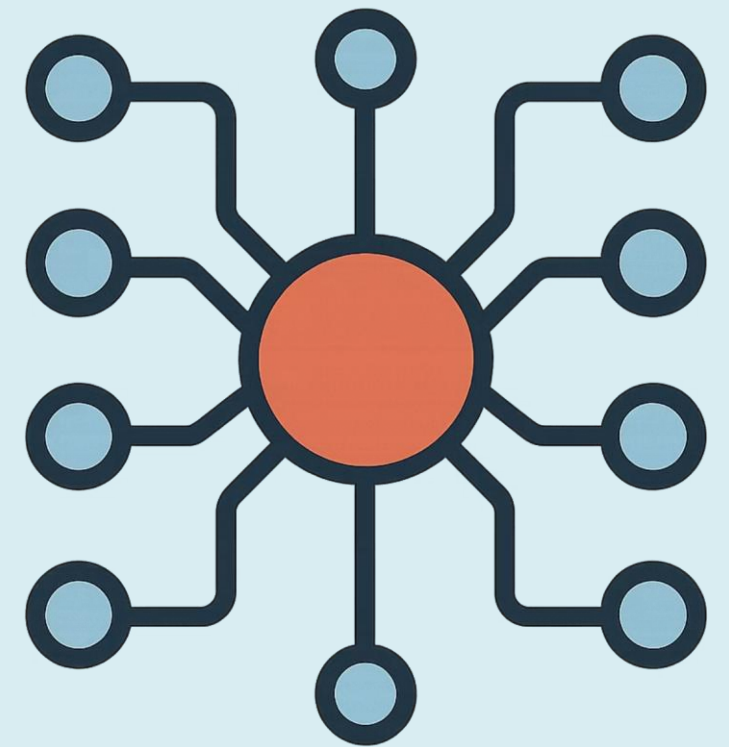


Retrieval-Augmented Generation (RAG)

Comment enrichir les LLM avec vos propres données ?

MODULE 4 – RAG

by ROMAIN ALFRED



Le point de départ

Ce qu'un LLM ne sait pas faire

Un grand modèle de langage (LLM) est souvent perçu comme une entité qui "sait des choses" ou qui "cherche des réponses". C'est une illusion utile, mais dangereuse en production.

En réalité, un LLM fait une seule chose : **prédire le prochain token le plus plausible** en fonction du contexte qu'on lui donne. Il ne consulte pas de base de données, n'effectue pas de requête et n'accède à aucune source externe.

✗ Ne cherche pas

Aucune requête vers l'extérieur

✗ Ne sait pas

Prédit du texte plausible

✗ Ne cite pas

Pas de sources vérifiables

Les 3 limites naturelles

1 Connaissances figées

Le modèle est entraîné jusqu'à une date précise (cutoff). Rien de plus récent n'est accessible.

2 Hallucinations

Il génère du texte vraisemblable, même faux. Sans ancrage factuel, il invente.

3 Pas de sources

Impossible de tracer l'origine d'une réponse sans mécanisme externe.

RAG = LLM + Moteur de Recherche Vectoriel

Le RAG (Retrieval-Augmented Generation) est une architecture qui **comble les lacunes du LLM** en lui fournissant, au moment de la question, les documents pertinents issus de *vos propres sources*. Le modèle ne mémorise plus — il lit à la demande.

Retriever

Moteur de recherche vectoriel qui trouve les documents les plus proches de la question posée

Contexte injecté

Les documents récupérés sont insérés dans le prompt avant d'être envoyés au LLM

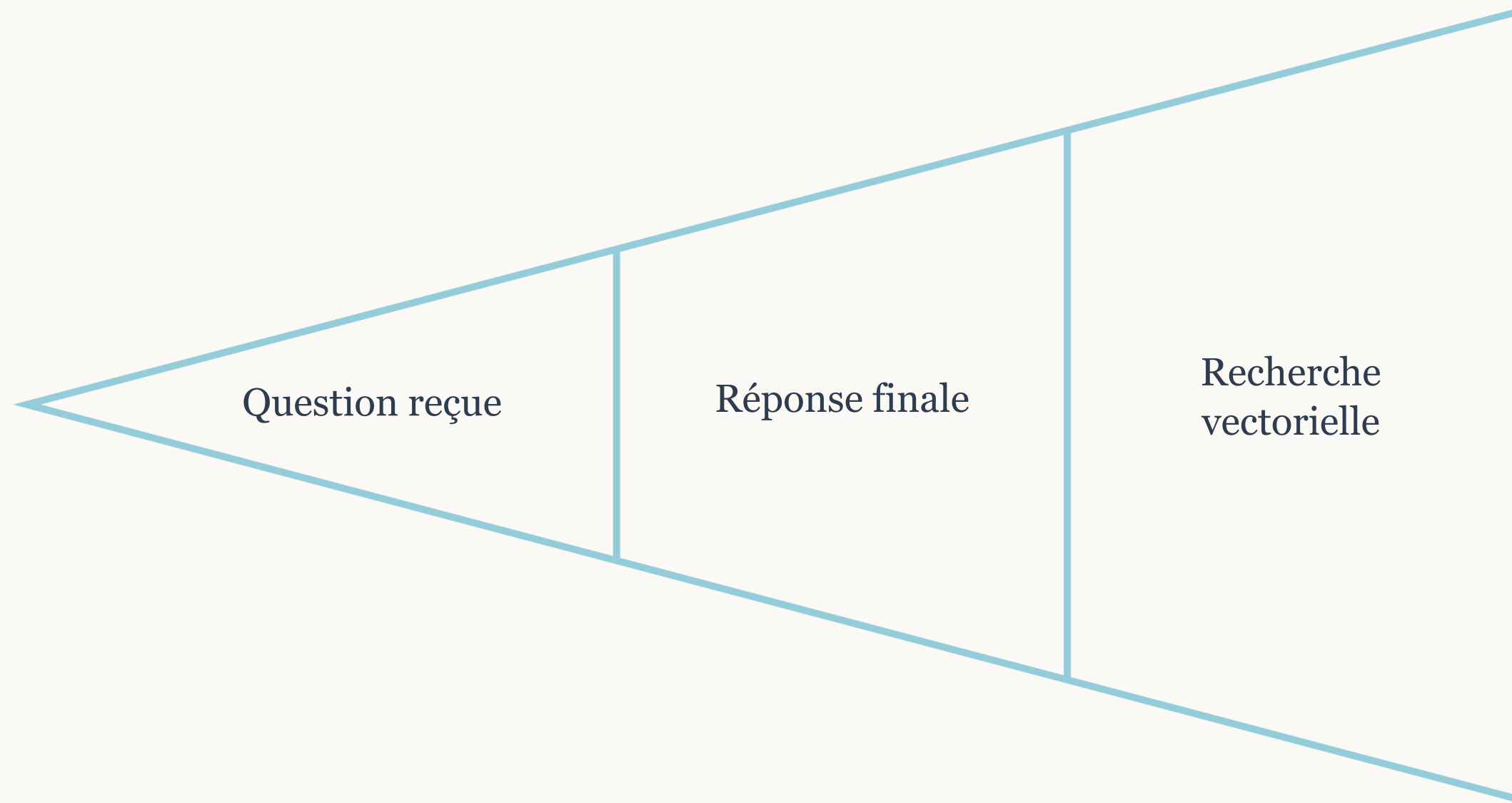
Generator (LLM)

Le modèle répond en s'appuyant sur ce contexte concret, réduisant drastiquement les hallucinations

- ✔ Le RAG permet d'utiliser des données privées, récentes ou propriétaires sans re-entraîner le modèle — une économie considérable de temps et de ressources.

Le Pipeline RAG de bout en bout

Comprendre le flux complet est essentiel avant de plonger dans les détails techniques. Voici les 6 étapes qui transforment une question en réponse ancrée dans vos données.



Ce pipeline s'exécute en temps réel à chaque requête utilisateur. La qualité du résultat final dépend de chaque étape — une mauvaise indexation ou un chunk mal calibré dégrade l'ensemble de la chaîne.

La Vector Database : cœur du système

Pourquoi pas une base SQL classique ?

Une base relationnelle effectue des **requêtes exactes** : elle cherche un mot, une valeur précise. Elle ne comprend pas le sens et est fragile lexicalement.

✗ SQL

"capitale de la France" ≠ "ville principale française" — aucun résultat !

✓ Vector DB

"capitale de la France" ≈ "ville principale française" — **match sémantique** !

La vector DB compare du **sens**, pas des mots. C'est la différence fondamentale.

Structure d'une entrée vectorielle

Champ	Rôle
id	Identifiant unique de l'entrée
embedding	Vecteur numérique (ex : 768 dimensions)
metadata	Source, date, type de document
text	Document original (chunk de texte)

❗ Chaque chunk de texte est converti en vecteur via un modèle d'embedding (ex : `text-embedding-ada-002` d'OpenAI ou `all-MiniLM` de Sentence Transformers).

Recherche de similarité & Indexation ANN

Le problème du scaling naïf

Comparer une requête à **1 million de vecteurs** en force brute = complexité $O(n)$. Inacceptable en production pour des temps de réponse sub-seconde.

La **similarité cosinus** est la métrique standard : elle mesure l'angle entre deux vecteurs, indépendamment de leur magnitude.

$$\text{sim}(A, B) = \frac{AB}{||A|| ||B||}$$

Solution : Approximate Nearest Neighbors (ANN)

1

HNSW

Hierarchical Navigable Small World — le standard moderne. Graphe multi-niveaux permettant une navigation ultra-rapide type "petit monde".

2

IVF

Inverted File Index — clustering des vecteurs en groupes. La recherche se limite aux clusters proches de la requête.

3

PQ

Product Quantization — compression des vecteurs pour réduire drastiquement la RAM nécessaire.

Les Vector Databases disponibles

Plusieurs solutions coexistent sur le marché, chacune avec ses forces. Le choix dépend de votre contexte : cloud vs on-premise, volume de données, et besoins en scalabilité.



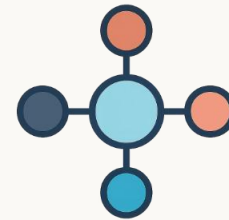
FAISS

Bibliothèque open-source de Meta. Idéale pour les **prototypes et environnements on-premise**. Ultra-rapide, mais sans interface native ni persistance automatique.



Pinecone

Service cloud managé. **Zéro configuration**, scalabilité automatique. Parfait pour une mise en production rapide. Modèle payant selon le volume.



Weaviate

Base vectorielle open-source avec **modules d'embedding intégrés**, API GraphQL, et support de métadonnées riches. Déployable en self-hosted ou cloud.



Chroma / Qdrant

Alternatives modernes très populaires dans l'écosystème LangChain/LlamaIndex. **Légères, rapides à intégrer**, avec bonne documentation.

Les Paramètres Clés du RAG

La performance d'un système RAG dépend de plusieurs paramètres à calibrer avec soin. Un mauvais réglage peut dégrader la pertinence des réponses même avec un excellent LLM.



Chunk Size

Taille des fragments de texte indexés. **Trop grand** = bruit dans le contexte. **Trop petit** = perte de cohérence. Typiquement entre 256 et 1024 tokens.



Overlap

Chevauchement entre chunks consécutifs pour éviter de couper une idée à mi-chemin. Généralement 10-20% de la taille du chunk.



Top-k Retrieval

Nombre de documents récupérés par requête. **k=3 à 5** est un bon point de départ. Un k trop élevé surcharge le prompt et dilue le signal pertinent.



Similarité Cosine

Métrique de distance entre vecteurs. Mesure l'**angle sémantique** entre la question et les chunks. Valeur entre -1 et 1 (plus proche de 1 = plus similaire).

⚠ Le calibrage de ces paramètres nécessite des tests empiriques sur votre domaine métier. Il n'existe pas de valeurs universelles optimales.

Exemple de Code : FAISS en Python

Voici un exemple minimal et commenté illustrant les 4 opérations fondamentales d'une vector database avec FAISS : créer un index, ajouter des vecteurs, lancer une recherche et récupérer les résultats.

```
import faiss
import numpy as np

# Dimension des embeddings (ex: 768 pour BERT)
d = 4

# Création d'un index L2 (distance euclidienne)
index = faiss.IndexFlatL2(d)

# Vecteurs à indexer (3 documents)
vectors = np.array([
    [1.0, 0.0, 0.0, 0.0], # doc A
    [0.9, 0.1, 0.0, 0.0], # doc B (proche de A)
    [0.0, 1.0, 0.0, 0.0], # doc C (différent)
]).astype('float32')

index.add(vectors) # Indexation

# Requête utilisateur (embeddinée)
query = np.array([[0.95, 0.05, 0.0, 0.0]]).astype('float32')

# Recherche des 2 voisins les plus proches
D, I = index.search(query, k=2)

print(I) # → [[1, 0]] : doc B puis doc A
```

Décryptage ligne par ligne

IndexFlatL2

Index brut sans approximation. Parfait pour tester, pas pour la production à grande échelle.

index.add()

Ajoute les vecteurs à l'index en mémoire. En prod, on utilise `IndexIVFFlat` ou `HNSW`.

index.search()

Retourne `D` (distances) et `I` (indices) des `k` voisins les plus proches.

Résultat `[[1, 0]]`

Le doc B (indice 1) est le plus proche de la requête, suivi du doc A (indice 0). Logique sémantiquement !

Synthèse & Points Clés à Retenir



Un LLM seul est limité

Connaissances figées, hallucinations, absence de sources — trois raisons fondamentales de ne pas l'utiliser seul en production métier.



Le RAG ancre les réponses

En injectant des documents pertinents dans le prompt, le RAG réduit les hallucinations et permet l'usage de données privées et récentes.



La vector DB est le moteur

Elle stocke les embeddings, les indexe efficacement (HNSW, IVF, PQ) et retrouve les plus proches sémantiquement via ANN.



Calibrer les paramètres

Chunk size, overlap, top-k et métrique de similarité sont les leviers à ajuster selon votre domaine et vos données.



Commencer avec FAISS

Pour prototyper rapidement, FAISS suffit. Pour la production, évaluez Pinecone, Weaviate ou Qdrant selon vos contraintes.

✔ Le RAG est aujourd'hui l'architecture de référence pour déployer des assistants IA fiables sur des données d'entreprise. Maîtriser son pipeline, c'est maîtriser l'IA en production.